



VICTORIA UNIVERSITY OF
WELLINGTON
TE HERENGA WAKA

School of Engineering and Computer Science
Te Kura Mātai Pūkaha, Pūrorohiko

PO Box 600
Wellington
New Zealand

Tel: +64 4 463 5341
Internet: office@ecs.vuw.ac.nz

The Title

Duy Nguyen

Supervisors: Dr Alvin Valera & Professor Winston
Seah

Submitted in partial fulfilment of the requirements for
Master of Computer Science.

Abstract

Load balancing in RPL using Machine learning

Acknowledgments

Thank you for Alvin and Winston for guiding me and helping me finish this project

Contents

1	Introduction	1
1.1	Contribution	2
1.2	Organisation	2
2	Background and Related Work	3
2.1	Routing Protocol for Low-Power and Lossy Networks (RPL)	3
2.1.1	Destination-Oriented Directed Acyclic Graph (DODAG)	3
2.1.2	Trickle Timer	4
2.1.3	Communication Patterns	5
2.2	Review of RPL Objective Functions	6
2.2.1	OF0 and MRHOF	6
2.2.2	Related Work	6
2.3	Contiki-NG and COOJA Simulator	7
2.3.1	Contiki	7
2.3.2	COOJA Simulator for Contiki-NG	7
3	Proposed Objective Function	9
3.1	Proposed Objective Function MLOF	9
3.2	Data Collection for Model Training	9
3.3	Regression Machine Learning Model	10
3.4	Feature Importance Analysis	10
3.5	Model Performance Evaluation	10
4	Performance Evaluation	11
4.1	Simulation Settings and Objectives	11
5	Performance Evaluation	13
5.1	QoS Evaluation	13
5.1.1	Packet Delivery Performance	13
5.1.2	Latency	13
5.1.3	CPU Utilisation	13
5.1.4	DODAG Stability	13
5.2	Routing Overhead	13
5.3	Routing Topology	13
5.4	Effect of parameter α	13
6	Conclusions	15
6.1	Future Work	15

Figures

Chapter 1

Introduction

Recent development in the Internet of Things (IoT) have driven a rapid increase the number of devices connected to the Internet, with growth continues to accelerate in many areas: health care, construction, building. These devices typically communicate wirelessly, exchanging data over radio frequency links instead of relying on wired connections, which allows flexible deployment across wide range of physical environments. However, many such deployments involve large number of constrained devices, which are bounded by low computational power, memory and energy (battery), to cover large geographic areas. Because the underlying network technologies are limited in range, multi-hop routing is often required. Such networks are categorised as Low-power and Lossy Networks (LLN) as links in LLN are susceptible to loss and frequent packet drops in addition to constraints on individual devices.

These limitations led the Internet Engineering Task Force (IETF), which is an international organisation of network designers and researcher, to form Routing over Low-power and Lossy networks (ROLL) working group to investigate the routing solution for LLNs. The working group first published RFC 5548 [4], RFC 5673 [13], RFC 5826 [3], RFC 5867 [10] to define the routing requirements for smart urban, industrial area, home automation and building automation applications respectively. From there, the ROLL working group had investigated the existing routing protocols Open Shortest Path First (OSPF), Intermediate System to Intermediate System (IS-IS), Ad hoc On-Demand Distance Vector (AODV), and Optimized Link State Routing Protocol (OLSR), to which they found out that they do not satisfy all of the routing requirements. This led to ROLL introducing a new routing protocol designed specifically for LLNs called IPv6 based routing protocol for low power and lossy networks (RPL), standardised in RFC 6550 [2].

At its core, RPL constructs a Destination-Oriented Directed Acyclic Graph (DODAG) to route packets from nodes toward a root, using objective function (OF) to calculate path cost and select parents during DODAG construction. Because of this, OFs determine how the traffic is distributed across DODAG

The protocol fell short when it comes to performing under high traffic as load balancing was not considered in the design. Furthermore, the OFs that ROLL propose only consider one metric, which might not be enough to capture the inherent conditions of the network. Moreover, most of the OFs are reactive in nature, meaning that would react after a route has degraded, leading to decrease in network quality and reliability.

In this project, we propose a new machine learning based OF that take load balancing into account and predict the link quality to choose the parents

1.1 Contribution

- Contribute a new objective function based on machine learning which focuses on improving delivery ratio
- Investigate the feature importance of different metrics in determine the delivery ratio quality

1.2 Organisation

The rest of this report is structured as follow

- Chapter 2 provides background information in relation to RPL and objective functions and related work in improving the objective functions
- Chapter 3 provides the motivation and overview the machine learning techniques and models used for our proposed objective function
- Chapter 4 shows the experiment and result from applying ML to OF

Chapter 2

Background and Related Work

This chapter provides a overview of RPL and load balancing objective function in RPL in general.

2.1 Routing Protocol for Low-Power and Lossy Networks (RPL)

RPL is an oriented distance vector routing protocol, operating on top of IEEE 802.15.4 PHY and MAC-layer and over 6LoWPAN (IPv6 over Low-Power Wireless Personal Area Networks). Its main target is Wireless Sensor Networks (WSNs), where multipoint-to-point is the predominant communication pattern. Routes in RPL are optimised for traffic to or from one or more roots in the topology, while maintaining reliable and energy efficient data collection. To sustain this, RPL is designed adapt to various network conditions, allowing nodes to switch route when route degradation is detected.

2.1.1 Destination-Oriented Directed Acyclic Graph (DODAG)

In principle, RPL organises nodes under a DODAG, with the sink node of the WSN often serving as the root of DODAG. When a packet need to be sent to the sink node, the packet traverse the path in DODAG until it reach the root node. The acyclic nature of DODAG ensures that the packet does not get stuck in endless circle.

There are four identifiers to define maintain a topology in RPL Table 2.1, with RPLInstanceID and DODAGID used to identify a single DODAG in the network.

Table 2.1: RPL DODAG identifiers

Identifier	Description
RPLInstanceID	The identity of a RPL instance, which may contain multiple DODAGs optimised for different OFs and applications.
DODAGID	Specifies a DODAG within a RPL instance.
DODAGVersionNumber	When a DODAG rebuild is needed, the root node increments the version number.
Rank	Defines the distance estimation of a node relative to the DODAG root.

During DODAG construction, each node selects parent from its pool of potential nodes that have path to the root through objective function, which translates one or more metrics

into rank and determines how to select the best parent. The equation of the rank is calculated as

$$Rank(N) = Rank(P) + Rank_Increase$$

where $Rank(N)$ is the rank of the current node, $Rank(P)$ is the rank advertised by a node and $Rank_Increase$ is determined by OF, which can be considered as the cost of choosing P as parent.

RPL offers flexibility by separating the core packet processing and forwarding from routing optimisation, leaving the OF to be specified depends on application to fit diverse requirements of LLNs, e.g. lowering energy usage, lowering latency,...

In addition, RPL introduces five types of control messages to form and control DODAG, as specified in ICMPv6 via RFC 6550 [2]

- DODAG Information Object (DIO): carries information that allow nodes to discover a RPL instance, select parents and maintain the DODAG.
- DODAG Information Solicitation (DIS): a request message when a node want to join a DODAG
- Destination Advertisement Object (DAO): the node advertise itself to the root node, indicating that it has join DODAG and is reachable
- Destination Advertisement Object Acknowledgement (DAO-ACK): the acknowledgement of the root, confirming the downward path to the node
- CC:

To begin forming DODAG, the root node starts sending DIO to its neighbouring nodes. Upon receiving DIO, a node decides whether to join the advertised DODAG or not. To prevent loops, a node only considers a DIO if it advertises a rank lower than its own rank. In certain implementations, a node would wait for several DIOs to build the pool of candidate parents before deciding. Afterward, the node computes the path cost of the corresponding link for each candidate and selects the candidate with the lowest path cost as its preferred parent. Once a parent has been selected, the node broadcasts its own DIO to advertise itself to its neighbours. This process is done until all the nodes have join the DODAG.

A node would send DAO message back to the root node after it has chosen a parent, Afterward, the nodes broadcast its DIO to advertise itself to the neighbour nodes. A node only consider the DIO if it has ranks lower than the rank advertised in the DIO. In the DAO-ACK mode, a node wait for the root node to send back DAO-ACK before confirming that it is reachable and advertise DIO to surrounding nodes

RPL also uses DIS when a new node want to join the network. Upon receiving DIS, nodes will unicast their DIO to the sending node and the sending node can decide whether to join the advertised DODAG and choose parents based on DIOs.

2.1.2 Trickle Timer

DIO is crucial in forming and maintaining DODAG, thus RPL also employs Trickle timer to regulate the emission of DIO. Trickle timer works by adjusting the sending interval dynamically based on the packets and network condition to ensure timely send while reducing redundant message transmission. The Trickle timer does this by evaluating the consistency of the network. When the network information are consistent, it exponentially increases the transmission interval of DIO. However, upon detecting inconsistencies, trickle timer reset its send interval to boost DIO transmission to spread the latest information to the DODAG.

Table 2.2: Trickle Timer Parameters

Parameter	Symbol	Description
Min interval length	I_{min}	The minimum sending interval between two messages
Max interval length	I_{max}	The maximum number of times that the sending interval can double
DIO redundancy constant	k	The number of consistent DIOs before suppresses its own DIO transmission

Table 2.2 shows three configuration parameters: I_{min} , I_{max} and k that Tricker timer to determine the send interval. The maximum sending interval time is:

$$I_{max_time} = I_{min} \times 2^{I_{max}}$$

Trickle timer set the initial sending interval I in the range of $[I_{min}, I_{max_time}]$, usually the initial interval is set to $I = I_{min}$. The trickle timer will select a random time t between $[I/2, I)$ as the sending interval. When Trickle detects a consistent message, it will increase the counter c . At the time t , if the value of c is less than the redundancy constant k , the trickle transmits the message, otherwise it would suppress the transmission. After the interval I , Trickle double the time of the interval up to I_{max_time} i.e. $I = \max(I \times 2, I_{max_time})$.

Upon detecting inconsistent messages, The interval I is reset to I_{min} .

In RPL specification, inconsistencies are defined as: These inconsistencies include join new DODAG version (e.g. new DODAGVersionNumber, new RPL instance, switching parent), when nodes detect loop and additional inconsistencies according to the implementation. However, the specification leaves the implementation adds more inconsistencies type when needed.

2.1.3 Communication Patterns

There are two types of communication patterns in RPL: upward route and downward route

Upward Route

In this approach, the upward routes follow the DODAG path to root as presented in the earlier section. A node would select one node as a preferred parent according the the rank calculated from OF

Downward Route

The downward are supported to P2MP and P2P route, in which the destination is not the root node. In this pattern, the node sends DAO to its preferred parent in a unicast manner

In this pattern, the RPL specification defines two mode: storing mode and non-storing mode. In storing mode, the downward routing tables are included in each nodes. However, in non-storing mode, the downward routing tables are stored only in the DODAG root. This means that P2P communication need to be forwarded to the root node, where the root would then formulate the routing path to the downward node.

The specification also opens option to specify DAO-ACK or not. A node would only consider reachable only after it has received the DAO-ACK.

2.2 Review of RPL Objective Functions

Up to now, the IETF ROLL working group has standardised only two objective functions: Objective Function Zero (OF0) [17] and Minimum Rank with Hysteresis Objective Function (MRHOF) [8].

2.2.1 OF0 and MRHOF

OF0 was developed to be a simple objective function, computing *Rank_Increase* as follow

$$Rank_Increase = (Rf \times Sp + Sr) \times Min_Rank_Increase$$

where *Rf* (Rank Factor) is a small positive integer used to scale the step of rank *Sp* (Step of Rank) is an integer value representing hop count in abstract sense; and *Sr* (Stretch of Rank) is used to inflate *Rank_Increase* for hysteresis preventing the OF from switching parents constantly. Certain implementation such as Contiki-NG simplify OF0 parameters by selecting $Rf = 1$, $Sp =$ and $Sr = 0$, which reduces *Rank_Increase* to a simple fixed per-hop increment of 256.

OF0 can be regarded as implicitly using hop count as routing metric. This mean that the traffic from OF0 follow the shortest path to root without considering any load balancing requirement. However, this also means that OF0 does not consider other link qualities such as latency, energy residual in its calculation as well as might choose worse off route but have the shortest hop count to root. This led to the introduction a more sophisticated MRHOF.

In contrast, MRHOF propose multiple thresholds call hysteresis before switching, in particular path cost threshold and time threshold. In path cost threshold, unless the potential node is better than the current parent by a threshold, it will stick with the current parent. However, after the time threshold, if that node is still better then the node will switch parent. This would allows a more stabilised DODAG. Additionally, MHROF use link metric ETX instead of hop count. Several metrics have been proposed to be used in objective functions. However, the specification of RPL leaves the choice of which metrics to use open to the implementation.

The default metric of MRHOF is Expected Transmission Count (ETX) metric. The main goal of ETX is to select routes with high end-to-end through put:

$$ETX = \frac{1}{D_t \times D_a}$$

where D_t is the probability that the receiving node receives the packet and D_a is the probability that the transmitting node receives acknowledgement packet is successful. So ETX is the average number of packets need to send and acknowledgement.

In essence, using ETX as metrics focus on more reliable route.

There are two thresholds in MRHOF, rank threshold and time threshold. The specification define *PARENT_SWITCH_THRESHOLD* as the minimum path cost improvement before deciding to switch. There is time threshold, meaning that if a node consistently perform better but not more than the ETX threshold, it still switch.

The specification also open the choices of the routing metrics to energy,... However, ETX is the more popular choice when it comes to MHROF.

Thanks to the flexibility of DIO header option.

2.2.2 Related Work

Various objective functions have been proposed, which can be categorised into 4 groups: single metric, composite and context aware.

In [9], the authors used queue utilisation metric in objective function, called QU-RPL to deal with dropped packets due to queue overflow. However, only queue metrics are taken into consideration, without mentioning latency.

A new objective function: OF-FL is presented in [7], which employs fuzzy logic with four metrics with the aim of supporting various application requirements.

Additionally, ... evaluates the performance of the standardised OFs and find performance of OF0 degrade upon increasing load

- Various objective functions have been proposed - Types of Objective function - Single metrics - Composite - Context aware

- Energy aware: [1] - CLRPL: [16]

In recent, there has been many attempts to employ machine learning into OFs. Despite that, most of the new application of ML in RPL focuses on reinforcement learning with little focus on supervised learning

In [6], the authorse puts Q-Learning to OF, award the model when a packet is trasmitted successfully and penalise when choosing a link that lead to packet loss. [5] utilised Random Weigth with six metrics (...) and [11], [15] proposed using CatBoost models with eight metrics to predict the probability if a packet is transmitted successfully.

[14] evaluate the performance of OF0 and MRHOF in COOJA Contiki

2.3 Contiki-NG and COOJA Simulator

2.3.1 Contiki

Contiki-NG is a lightweight operating system for resource-constrained devices in WSNs and IoT.

which was built on the Contiki.

The Swedish Institute of Computer Science (SICS) released the first version of Contiki in 2003 Contiki was created by Adam Dunkels in 2002, which supports various protocol for IoT devices and have been support from various research instutitions Later on, ContikiMAC was introduce as the sole duty cycle MAC layer for RPL.

However, recently a new Contiki cross-platform has been released on Nov 2017 named Contiki-NG 4.0, which licensed under the 3-clause BSD license. Contiki-NG concentrates on the Next-Generation IoT devices such as ARM Cortex M3 and other 32-bit MCUs and on reliable and secure standard protocols like IPv6/6LoWPAN, 6TiSCH, RPL, and CoAP [12].

In Contiki-NG, RPL implementation was rewritten as RPL-lite while the original version of RPL in Contiki is kept as RPL-classic. RPL-lite removes certain features RPL based on year of experience, keeping the supports one single DAG at a time, one single "RPL instance", and only the non-storing mode of operation.

Additionally, ContikiMAC was removed in favor of TSCH implemetation, thus making TSCH the default for duty cycle MAC implementation.

2.3.2 COOJA Simulator for Contiki-NG

The Cooja simulator is widely used among RPL OF researches

Cooja is an elastic Java-based simulator dedicated for WSNs based on Contiki Operating System. Cooja is capable of simulating different node types, including simulated hardward (consisting of clock cycle and MCU) and software based simulation (which skip the internal working of an MCU).

In this regard, Cooja is a tool for real hardware simulation.

depends on the mote type, provides three mote types out-of-the-box: Sky, Z1 and Cooja where Sky and Z1 are emulated hardware for TelosB and Zolertia Z1 and Cooja is the software simulation. Table 2.3 summarises the board specification.

Table 2.3: Supported mote hardware specifications in COOJA simulator

Specification	Zolertia Z1	TelosB
Microcontroller	MSP430F2617 (2nd gen)	Texas Instruments MSP430
CPU	16-bit RISC @ 16 MHz	8 MHz
RAM	8 KB	10 KB
Flash memory	92 KB	48 KB
Transceiver	CC2420, IEEE 802.15.4	Chipcon, IEEE 802.15.4
Frequency band	2.4 GHz	2.4 GHz
Data rate	250 kbps	250 kbps

Thus to maximise the computing power for machine learning model, Z1 is the most suitable mote for our need.

This platform is a virtual platform, used by the Cooja platform to run Contiki-NG as ‘Cooja motes’. It compiles Contiki-NG as a native process, and connects directly all hardware accesses to the Cooja simulator. Unlike emulations, simulation with Cooja motes are not perfectly time-accurate, but the timing is still good enough to run the MAC layers CSMA and TSCH

Chapter 3

Proposed Objective Function

3.1 Proposed Objective Function MLOF

Most of the OFs have been using one metrics, in which the objective function in a sense is greedy, selecting nearby neighbours with the best link quality without regarding the overall performance of the whole route.

In this sense, I propose using a ML technique to analyse the feature importance of quality of a link. Once we know the most important routing metrics that effect the quality of a link, we can design an efficient objective function based on that result

The proposed ML-based OFs in the related work section are reinforcement learning by design. The one using supervised learning however only predict the success transmission probability of one packet, missing the whole picture.

In this OF, we proposed using machine learning model to predict the packet delivery ratio (PDR) and the use the predicted PDR to calculate the rank as follow

$$Rank_Increase = \alpha \times Predicted_PDR + (1 - \alpha) \times ETX$$

We include ETX as a measure to control the prediciton of the model incase of wild prediction, which could lead to unstable DODAG and reduce the network performance.

Addition to this, we also measured the overhead of introducing machine learning model to the OFs

Metrics of interest - End-to-End latency: - Hop Count - Parent switch: - CPU util: in this metrics, we want to see the the CPU util to be more even depends on the hop count. Due to the energy usage are similar to each other in CSMA MAC, we opted for CPU util instead.

Gather metrics, processing data -¿ Train model -¿ Evaluation and Porting to C

3.2 Data Collection for Model Training

We utilise 11 routing metrics for the feature importance analysis ETX: Hop count: RSSI:

PPM: the total packet transmission per minute. These include retransmission and packets from children nodes CPU util: The percentage of active CPU over the total time Drop rate: the number of successful transmission per one drop. The higher the value the lower the drop ratio

Is new: which is the metric to differentiate between the current parent or new parent, based on the way that we collect the metrics

In our evaluation, the nodes closer to root node would have higher CPU util since they have to process more packets from children node, thus also higher PPM and potentially higher Drop rate.

More over, we also include parent PPM, parent CPU util and parent neighbour counts in the DIO to advertise to the child. Possibly to link the load increase of the current node onto the selected parent

And finally choose 6 most importance metrics to train the model for PDR prediction

The routing metrics are collected on two circumstances: on switching parent and on receiving the DIO update from its current parent and decide not to switch parent. Because of this, we add is new to the training data feature to differentiate between current parent or new parent as current parent also include the load of the current node.

Afterward, the training label PDR is calculated within a scope of a node by calculating TX / RX between the current row and next data input time the current row and the next row of the same node. We want the model to be able to predict if this parent is selected, what would be the potential PDR is.

Additionally, we filter the chunk having less than 30 seconds to reduce the noise in the training dataset.

The data is collected from simulation in COOJA with parameters include in Table ...

3.3 Regression Machine Learning Model

The advance of machine learning has led the wide applications in IoT devices.

KNNR: works by comparing a node with the surrounding nodes to predict a value. In a sense, KNNR is considered a lazy model, in which it remembers all training data, thus result in a heavy model

Gaussian Process Regressor, similar to SVM. However, in [?], the authors analyse GPR and conclude that the memory requirement of GPR is $O(n^2)$ while the computation cost is $O(n^3)$, which is quite computational and memory expensive for constrained devices

Deep learning is recently a new advances and has achieve in terms of matching the complex patterns and data. However, due to the constrained environment of the working MCU which has several KB of RAM and ROM. I rule out this.

Random forest and Decision Tree operates on similar algorithm. In tree base algorithm, each value is split into left and right, following the path to reach the end. In Random Forest, multiple trees are used to prevent overfitting and provide better performance. However, in my experiments, the performance of random forest and decision tree are quite similar to each other. Thus I remove random forest from list of considered models as they are much heavier than decision tree.

In the end, we decide to proceed with Random Forest, Linear Regression and Support Vector Regression due to the design and computation simplicity to fit in with the constrained of the devices that we operate on.

3.4 Feature Importance Analysis

3.5 Model Performance Evaluation

Three models

Quantisation

Size of the models

Chapter 4

Performance Evaluation

4.1 Simulation Settings and Objectives

The OFs are simulated in COOJA, using Z1 mote Table ... list the running parameters for the experiment

We use CSMA

Chapter 5

Performance Evaluation

In this section, we analyse the measurements of MLOF against OF0 and MRHOF from result obtained from 120 simulations in COOJA simulator

5.1 QoS Evaluation

5.1.1 Packet Delivery Performance

PDR

5.1.2 Latency

Latency

5.1.3 CPU Utilisation

CPU util

5.1.4 DODAG Stability

Parent switch

5.2 Routing Overhead

In the model, we added 7 fields, which increase the overhead
Memory analysis,

5.3 Routing Topology

5.4 Effect of parameter α

Chapter 6

Conclusions

The conclusions are presented in this Chapter.
Limitation

6.1 Future Work

Bibliography

- [1] ABREU, C., RICARDO, M., AND MENDES, P. Energy-aware routing for biomedical wireless sensor networks. *Journal of Network and Computer Applications* 40 (Apr. 2014), 270–278.
- [2] ALEXANDER, R., BRANDT, A., VASSEUR, J. P., HUI, J., PISTER, K., THUBERT, P., LEVIS, P., STRUIK, R., KELSEY, R., AND WINTER, T. RPL: IPv6 Routing Protocol for Low-Power and Lossy Networks. Request for Comments RFC 6550, Internet Engineering Task Force, Mar. 2012.
- [3] BRANDT, A., BURON, J., AND PORCU, G. Home Automation Routing Requirements in Low-Power and Lossy Networks. RFC 5826, Apr. 2010.
- [4] DOHLER, M., WATTEYNE, T., WINTER, T., AND BARTHEL, D. Routing Requirements for Urban Low-Power and Lossy Networks. RFC 5548, May 2009.
- [5] DUENAS SANTOS, C. L., ASTUDILLO LEÓN, J. P., MEZHER, A. M., CARDENAS BARRERA, J., MENG, J., AND CASTILLO GUERRA, E. RPL+: An Improved Parent Selection Strategy for RPL in Wireless Smart Grid Networks. In *Proceedings of the 19th ACM International Symposium on Performance Evaluation of Wireless Ad Hoc, Sensor, & Ubiquitous Networks* (2022), PE-WASUN '22, Association for Computing Machinery, pp. 75–82.
- [6] DUENAS SANTOS, C. L., MEZHER, A. M., ASTUDILLO LEÓN, J. P., CARDENAS BARRERA, J., CASTILLO GUERRA, E., AND MENG, J. Q-RPL: Q-Learning-Based Routing Protocol for Advanced Metering Infrastructure in Smart Grids. 4818.
- [7] GADDOUR, O., KOUBÂA, A., BACCOUR, N., AND ABID, M. OF-FL: QoS-aware fuzzy logic objective function for the RPL routing protocol. In *2014 12th International Symposium on Modeling and Optimization in Mobile, Ad Hoc, and Wireless Networks (WiOpt)* (2014), pp. 365–372.
- [8] GNAWALI, O., AND LEVIS, P. The Minimum Rank with Hysteresis Objective Function. RFC 6719, Sept. 2012.
- [9] KIM, H.-S., PAEK, J., AND BAHK, S. QU-RPL: Queue utilization based RPL for load balancing in large scale industrial applications. In *2015 12th Annual IEEE International Conference on Sensing, Communication, and Networking (SECON)* (June 2015), pp. 265–273.
- [10] MARTOCCI, J., MIL, P. D., RIOU, N., AND VERMEYLEN, W. Building Automation Routing Requirements in Low-Power and Lossy Networks. RFC 5867, June 2010.
- [11] MEZHER, A. M., DUEÑAS SANTOS, C. L., REBOLLO-MONEDERO, D., CÁRDENAS-BARRERA, J., AGUILAR IGARTUA, M., MENG, J., AND CASTILLO GUERRA, E. GNB-RPL: Gaussian Naïve Bayes for RPL Routing Protocol in Smart Grid Communications.

In *Proceedings of the 19th ACM International Symposium on QoS and Security for Wireless and Mobile Networks* (2023), Q2SWinet '23, Association for Computing Machinery, pp. 53–60.

- [12] OIKONOMOU, G., DUQUENNOY, S., ELSTS, A., ERIKSSON, J., TANAKA, Y., AND TSIFTES, N. The Contiki-NG open source operating system for next generation IoT devices. *SoftwareX* 18 (2022), 101089.
- [13] PISTER, K., THUBERT, P., DWARS, S., AND PHINNEY, T. Industrial Routing Requirements in Low-Power and Lossy Networks. RFC 5673, Oct. 2009.
- [14] PRADESKA, N., WIDYAWAN, NAJIB, W., AND KUSUMAWARDANI, S. S. Performance analysis of objective function MRHOF and OF0 in routing protocol RPL IPV6 over low power wireless personal area networks (6LoWPAN). In *2016 8th International Conference on Information Technology and Electrical Engineering (ICITEE)* (Oct. 2016), pp. 1–6.
- [15] SANTOS, C. L. D., MEZHER, A. M., LEÓN, J. P. A., BARRERA, J. C., GUERRA, E. C., AND MENG, J. ML-RPL: Machine Learning-Based Routing Protocol for Wireless Smart Grid Networks. *IEEE Access* 11 (2023), 57401–57414.
- [16] TAGHIZADEH, S., BOBARSHAD, H., AND ELBIAZE, H. CLRPL: Context-Aware and Load Balancing RPL for Iot Networks Under Heavy and Highly Dynamic Load. 23277–23291.
- [17] THUBERT, P. Objective Function Zero for the Routing Protocol for Low-Power and Lossy Networks (RPL). RFC 6552, Mar. 2012.